【扔物线 Compose 课程讲义】5.5-DrawModifier 的工作原理和对绘制的精细影响

关于讲义

讲义是对课程内容中**所有结论的集中汇总**。所以在看完每节课程之后，迅速来看一遍讲义，对课程内容做回顾，会有非常好的吸收效果。

面试前不要背讲义

在面试之前，通过快速浏览一遍讲义来确认所有知识都没有忘记，是个很好的备战方式。

但**不要用【背讲义上的知识点】的方式来备战**。最有效的浏览方式是：

浏览讲义中的所有内容，并：

- 看到「懂，并且记得背后的原因（或技术细节）」的，直接跳过；
- 看到「懂，但细节已经忘了」的，快速回看当节课程，把知识捡起来；
- 看到「已经不太懂 / 完全忘了」的，认真重看当节课程，把知识学回来。

疑问或意见

如果你发现讲义或课程有错误，或有别的什么意见，可以向任何一个你认识的助教反馈。

如果认可我们的课程，还请给个好评！ 评价链接：https://rengwuxian.com/cp

基本写法

扔物线学堂 - 秉心(1437182)

常用写法 Kotlin 复制代码

```kotlin
// 实用写法
Modifier.drawWithContent {
  // 被内部盖住
  drawContent()
  // 盖住内部
}

// 本质上等价于
Modifier.then(object : DrawModifier {
  override fun ContentDrawScope.draw() {
    drawContent()
  }
})
```

工作原理

- 在 `modifier.set()` 内部的遍历过程中，会被塞入这个 `DrawModifier` 右边最近的 `LayoutModifier` 所属的 `ModifiedLayoutNode`，因此它的绘制范围由它最右边的 `LayoutModifier` 决定；
- 一个 `Modifier` 链条上的多个 `DrawModifier`，会以「左边优先于右边」的顺序进行调用，所以「左边的」总是包含「右边的」；
- `DrawModifier` 不会主动绘制内部（右边）的 `DrawModifier`，但 `drawContent()` 函数可以进行这项工作，所以需要主动调用这个函数来确保绘制的逻辑链条不断开。